

Chapter 6

arrays, vectors, Ranges and Functional-Style Programming

Learning Objectives

- Use C++ standard library class template array
- Declare arrays, initialize arrays and refer to the elements of arrays.
- Use the range-based for statement to reduce iteration errors.
- Pass arrays to functions.
- Perform common array manipulations.
- Visualize an array's data.
- Sort array elements in ascending order.
- Quickly determine whether a sorted array contains a specific value using the high-performance `binary_search` function.
- Declare and manipulate multidimensional arrays.
- Use C++20's ranges with functional-style programming.

Introduction

- This chapter introduces **data structures**—collections of related data items.
- The C++ standard library refers to data structures as **containers**. We discuss two containers:
 - fixed-size **arrays** and
 - resizable vectors that can grow and shrink dynamically at execution time.
- To use them, you must include the `<array>` and `<vector>` headers, respectively.

Introduction (Cont.)

- After discussing how arrays are declared, created and initialized, we demonstrate various array manipulations.
 - We show how to search arrays to find particular items and sort arrays to put their data in ascending order.
 - We show that attempting to access data outside an array's or vector's bounds might cause an exception—a runtime indication that a problem occurred.
 - Then we use exception handling to resolve (or handle) that exception.

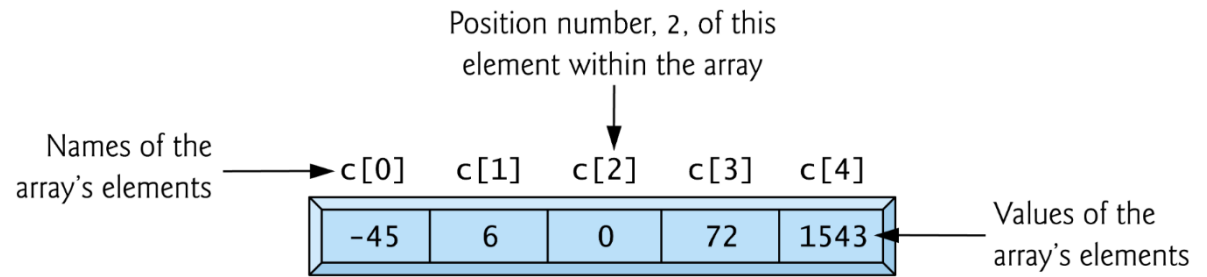
Introduction (Cont.)

- Like many modern languages, C++ offers “functional-style” programming features.
 - These can help you write more concise code that’s less likely to contain errors and easier to read, debug and modify.
 - Functional-style programs also can be easier to parallelize to improve performance on today’s multi-core processors.
 - We introduce functional-style programming with C++20’s new `<ranges>` library.
- Finally, we continue our Objects-Natural presentation with a case study that creates and manipulates objects of the C++ standard library’s class template `vector`.

Introduction.

- Fully Qualified Standard Library Names
 - Since Chapter 2, we've included "using namespace std;" in each program, so you did not need to qualify every C++ standard library feature with "std::".
 - In larger systems, using directives can lead to subtle, difficult-to-find bugs.
 - Going forward, we'll fully qualify most identifiers from the C++ standard library.

arrays



- An array's elements (data items) are arranged contiguously in memory.
- The above diagram shows an integer array called `c` that contains five elements.
- One way to refer to an array element is to specify the **array name** followed by the element's **position number** in **square brackets** (`[]`).
 - The position number is more formally called an index or subscript.
 - The first element has the index 0 (zero).
 - Thus, the elements of array `c` are `c[0]` (pronounced "c sub zero") through `c[4]`.
 - The value of `c[0]` is -45, `c[2]` is 0 and `c[4]` is 1543.

arrays.

- The **highest index** (in this case, 4) is always one less than the array's number of elements (in this case, 5).
- Each array knows its own size, which you can get via its **size member function**, as in `c.size()`
- An index must be an **integer expression**. The brackets enclosing an index are an operator with the **second highest level of precedence**. The result of that operator is an lvalue—it can be used on the left side of an assignment, just as a variable name can.
- For example, the following statement replaces `c[4]`'s value:
`c[4] = 87;`

Declaring arrays

- To specify an array's element type and number of elements, use a declaration of the form
`std::array<type, arraySize> arrayName;`
 - The notation <type, arraySize> indicates that array is a **class template**.
 - Like function templates, the compiler can use class templates to **create class template specializations** for various types.
 - ◆ E.g., an array of five ints, an array of seven doubles or an array of 100 Employees.
 - The compiler reserves the appropriate amount of memory based on the type and arraySize.
- To tell the compiler to reserve five elements for integer array c, use this declaration:
`std::array<int, 5> c; // c is an array of 5 int values`

Initializing array Elements in a Loop

- The next program declares five-element integer array values (line 7).
- Line 4 includes the `<array>` header containing the class template array's definition.

```
1 // Initializing an array's elements to zeros and printing the array.
2 #include <format>
3 #include <iostream>
4 #include <array>
5
6 int main() {
7     std::array<int, 5> values; // values is an array of 5 int values
```

Initializing array Elements in a Loop (Cont.)

- Assigning Values to array Elements in a Loop
 - Lines 10–12 use a for statement to assign 0 to each array element.
 - Like other non-static local variables, array elements are not implicitly initialized to zero but elements of static arrays are.
 - In a loop that processes array elements, ensure that the loop's condition prevents accessing elements outside the array's bounds.
 - Later we will present the range-based for statement, which provides a safer way to process every element of an array.

```
9  
10 // initialize elements of array values to 0  
11 for (size_t i{0}; i < values.size(); ++i) {  
12     values[i] = 0; // set element at location i to 0  
    }
```

Initializing array Elements in a Loop (Cont.)

- Type `size_t`

- Lines 10, 17 and 24 declare each loop's control variable as type `size_t`.
- The C++ standard specifies that `size_t` is an **unsigned integral type** that can represent the size of any object.
- Use this type for any variable representing an **array's size or indices**.
- Type `size_t` is defined in the `std` namespace and is in header `<cstddef>`, which is typically included for you by other standard library headers that use `size_t`.
 - ◆ If you compile a program and receive errors indicating `size_t` is not defined, add `#include <cstddef>` to the program.

```
9      // initialize elements of array values to 0
10     for (size_t i{0}; i < values.size(); ++i) {
11         values[i] = 0; // set element at location i to 0
12     }
```

Initializing array Elements in a Loop (Cont.)

- Displaying the array Elements

- The first output statement (line 14) displays the column headings for the rows printed in the subsequent for statement (lines 17–19).
- These output statements use the C++20 [text-formatting features](#) to output the array in tabular format.

```
14      std::cout << std::format("{:>7}{:>10}\n", "Element", "Value");
15
16      // output each array element's value
17      for (size_t i{0}; i < values.size(); ++i) {
18          std::cout << std::format("{:>7}{:>10}\n", i, values[i]);
19      }
20
21      std::cout << std::format("\n{:>7}{:>10}\n", "Element", "Value");
22
23      // access elements via the at member function
24      for (size_t i{0}; i < values.size(); ++i) {
25          std::cout << std::format("{:>7}{:>10}\n", i, values.at(i));
26      }
```

Element	Value
0	0
1	0
2	0
3	0
4	0

Element	Value
0	0
1	0
2	0
3	0
4	0

Initializing array Elements in a Loop (Cont.)

- Avoiding a Security Vulnerability: Bounds Checking for array Indices
 - When you use the `[]` operator to access an array element (as in lines 10–12 and 17–19), C++ provides no automatic array bounds checking to prevent you from referring to an element that does not exist.
 - ◆ Thus, an executing program can “walk off” either end of an array without warning.

```
14      std::cout << std::format("{:>7}{:>10}\n", "Element", "Value");
15
16      // output each array element's value
17      for (size_t i{0}; i < values.size(); ++i) {
18          std::cout << std::format("{:>7}{:>10}\n", i, values[i]);
19      }
```

Element	Value
0	0
1	0
2	0
3	0
4	0

Initializing array Elements in a Loop (Cont.)

- Avoiding a Security Vulnerability: Bounds Checking for array Indices
 - Class template array's `at` member function, however, performs **bounds checking**.
 - ◆ Lines 24–26 demonstrate accessing elements' values via the `at` member function.
 - You also can assign to array elements by using `at` on the left side of an assignment, as in `values.at(0) = 10;`

```
23 // access elements via the at member function
24 for (size_t i{0}; i < values.size(); ++i) {
25     std::cout << std::format("{:>7}{:>10}\n", i, values.at(i));
26 }
```

Element	Value
0	0
1	0
2	0
3	0
4	0

Initializing array Elements in a Loop (Cont.)

- Avoiding a Security Vulnerability: Bounds Checking for array Indices
 - Line 29 attempts to access an element outside the array's bounds.
 - When the member function `at` encounters an out-of-bounds index, it generates a runtime error known as an exception.

```
28 // accessing an element outside the array's bounds with at
29 values.at(10); // throws an exception
30 }
```

terminate called after throwing an instance of
what(): array::at: __n (which is 10) >= _Nm (which is 5)
Aborted

Initializing array Elements in a Loop (Cont.)

- Avoiding a Security Vulnerability: Bounds Checking for array Indices

- In this program, which we compiled with GNU g++ and ran on Linux, line 29 resulted in the following runtime error message then the program terminated:

terminate called after throwing an instance of
what(): array::at: __n (which is 10) >= _Nm (which is 5)
Aborted

- ◆ This error message indicates that the at member function (array::at) checks whether a variable named __n (which is 10) is greater than or equal to a variable named _Nm (which is 5).
- ◆ In this case, the index is out of bounds.
- ◆ In GNU's implementation of array's at member function, __n represents the element's index, and _Nm represents the array's size.

```
28 | // accessing an element outside the array's bounds with at  
29 | values.at(10); // throws an exception  
30 | }
```

Initializing array Elements in a Loop.

- Avoiding a Security Vulnerability: Bounds Checking for array Indices
 - Allowing programs to read from or write to array elements outside the bounds of arrays are common security flaws.
 - Reading from out-of-bounds array elements can cause a program to crash or even appear to execute correctly while using bad data.
 - Writing to an out-of-bounds element (known as a **buffer overflow**) can corrupt a program's data in memory and crash a program.
 - ◆ Attackers can sometimes exploit buffer overflows by over-writing the program's executable code with malicious code.

Initializing an array with an Initializer List

- The next program uses initializer lists to initialize arrays.
- The elements of an array also can be initialized in the array declaration by following the array name with a **brace-delimited comma-separated list of initializers**.
- Line 7 explicitly tells the compiler the array's element type (int) and size (5).

```
1 // Initializing an array in a declaration.
2 #include <format>
3 #include <iostream>
4 #include <array>
5
6 int main() {
7     std::array<int, 5> values{32, 27, 64, 18, 95}; // braced initializer
8
9     // output each array element's value
10    for (size_t i{0}; i < values.size(); ++i) {
11        std::cout << std::format("{} ", values.at(i));
12    }
```

32 27 64 18 95

Initializing an array with an Initializer List (Cont.)

- Line 17 lets the compiler determine the array's element type (double) from the initializer list's values and the array's size (4) from the number of initializers.
- This capability is known as **class template argument deduction (CTAD)** and simplifies your code when you know the container's initial element values in advance.

```
16 // using class template argument deduction to determine values2's type
17 std::array values2{1.1, 2.2, 3.3, 4.4};
18
19 // output each array element's value
20 for (size_t i{0}; i < values2.size(); ++i) {
21     std::cout << std::format("{} ", values2.at(i));
22 }
23
24 std::cout << '\n';
25 }
```

1.1 2.2 3.3 4.4

Initializing an array with an_INITIALIZER List (Cont.)

- Fewer Initializers Than array Elements
 - If there are fewer initializers than array elements, the remaining array elements are value initialized—fundamental numeric types are set to 0, bools are set to false, and as we'll see in Chapter 9, objects receive the default initialization specified by their class definitions.
 - For example, the following initializes an int array's elements to zero
`std::array<int, 5> values{}; // initialize elements to 0`
because there are fewer initializers (none in this case) than array elements.
 - This technique also can be used to “reinitialize” an existing array's elements at execution time, as in
`values = {}; // set all elements of values to 0`

Initializing an array with an Initializer List.

- More Initializers Than array Elements
 - When you explicitly state the array's size and use an initializer list, the number of initializers must be less than or equal to the size.
 - The array declaration
`std::array<int, 5> values{32, 27, 64, 18, 95, 14};`
causes a compilation error because there are six initializers and only five array elements.

Range-Based for Statement

- It's common to process all the elements of an array.
- The range-based for statement allows you to do this without using a counter.
- When processing all elements of an array, use the range-based for statement to ensure that your code does not “**step outside**” the array's bounds.
- The next program uses the range-based for to display an array's contents (lines 11–13 and 22–24) and multiply each element's value by 2 (lines 16–18).

Range-Based for Statement (Cont.)

- Using the Range-Based for to Display an array's Contents
 - The range-based for statement simplifies the code for iterating through an array.
 - Line 11 can be read as “**for each item in items,**” perform some work.
 - In each iteration, the loop assigns the next items element to the const int reference item, then executes the loop's body—item refers to one element's value in items.

```
1 // Using range-based for.
2 #include <format>
3 #include <iostream>
4 #include <array>
5
6 int main() {
7     std::array items{1, 2, 3, 4, 5}; // type inferred as array<int, 5>
8
9     // display items before modification
10    std::cout << "items before modification: ";
11    for (const int& item : items) { // item is a reference to a const int
12        std::cout << std::format("{} ", item);
13    }
```

items before modification: 1 2 3 4 5

Range-Based for Statement (Cont.)

- Using the Range-Based for to Display an array's Contents
 - In the range-based for's header, you declare a **range variable** to the left of the colon (:) and specify an **array's name** to the right.
 - ◆ Here we declare the range variable item as a const reference.
 - ◆ Recall that a reference is an alias for another variable in memory—in this case, one of the array's elements.
 - Declaring a range variable as a **const reference** can improve performance.
 - ◆ A reference prevents the loop from copying each value into the range variable. This is particularly important when manipulating large objects.

```
9 // display items before modification
10 std::cout << "items before modification: ";
11 for (const int& item : items) { // item is a reference to a const int
12     std::cout << std::format("{} ", item);
13 }
```

Range-Based for Statement (Cont.)

- Using the Range-Based for to Modify an array's Contents
 - Lines 16–18 use a range-based for statement to multiply each element of items by 2.
 - ◆ In line 16, the range variable's declaration indicates that item is an int&—that is, a reference to an int.
 - ◆ This is not a const reference, so any change you make to item changes the corresponding array element's value.

```
15 // multiply the elements of items by 2
16 for (int& item : items) { // item is a reference to an int
17     item *= 2;
18 }
19
20 // display items after modification
21 std::cout << "\nitems after modification: ";
22 for (const int& item : items) {
23     std::cout << std::format("{} ", item);
24 }
```

items after modification: 2 4 6 8 10

Range-Based for Statement (Cont.)

- C++20 Range-Based for with Initializer

- C++20 adds the range-based for with initializer statement.
- The initializer in line 28 defines runningTotal and sets it to 0, then lines 28–32 calculate the running total of items' elements.
- The variable runningTotal exists only until the loop terminates.

```
26 // sum elements of items using range-based for with initialization
27 std::cout << "\n\ncalculating a running total of items' values:\n";
28 for (int runningTotal{0}; const int& item : items) {
29     runningTotal += item;
30     std::cout << std::format("item: {}; running total: {}\n",
31                             item, runningTotal);
32 }
33 }
```

```
calculating a running total of items
item: 2; running total: 2
item: 4; running total: 6
item: 6; running total: 12
item: 8; running total: 20
item: 10; running total: 30
```

Range-Based for Statement (Cont.)

- Avoiding Indices

- In most cases, a range-based for statement can be used instead of the counter-controlled for statement.
- For example, totaling the integers in an array requires access only to the element values—their index positions in the array are irrelevant.

Range-Based for Statement (Cont.)

- External vs. Internal Iteration

- In this program, lines 11–13 are functionally equivalent to the following counter-controlled iteration:

```
for (size_t counter{0}; counter < items.size(); ++counter) {  
    std::cout << std::format("{} ", items[counter]);  
}
```

- ◆ This style of iteration is known as **external iteration** and is **error-prone**.
- ◆ As implemented, this loop requires a control variable (counter) that the code mutates (modifies) during each loop iteration.
- ◆ Every time you write code that modifies a variable, it's possible to introduce an error into your code.

```
9      // display items before modification  
10     std::cout << "items before modification: ";  
11     for (const int& item : items) { // item is a reference to a const int  
12         std::cout << std::format("{} ", item);  
13     }
```

Range-Based for Statement.

- External vs. Internal Iteration
 - There are several opportunities for error in the preceding code. For example, you could:
 - ◆ initialize the for loop's control variable counter incorrectly,
 - ◆ use the wrong loop-continuation condition or
 - ◆ increment the control variable counter incorrectly.
 - Each of these could result in accessing elements outside the array items' bounds.
 - The **range-based for** statement uses **internal iteration**, hiding the iteration details from you.
 - ◆ You specify what array the range-based for should process, and it knows how to get each value from the array and stop iterating when there are no more values.

Calculating array Element Values and an Intro to constexpr

- The next program sets the elements of a 5-element array named values to the even integers 2, 4, 6, 8 and 10 (lines 12–14) and prints the array (lines 17–19).

```
1 // Set array values to the even integers from 2 to 10.
2 #include <format>
3 #include <iostream>
4 #include <array>
5
6 int main() {
7     // constant can be used to specify array size
8     constexpr size_t arraySize{5}; // must initialize in declaration
9
10    std::array<int, arraySize> values{}; // array values has 5 elements
11
12    for (int i{0}; i < values.size(); ++i) { // set the values
13        values.at(i) = 2 + 2 * i;
14    }
15
16    // output contents of array values in tabular format
17    for (const int& value : values) {
18        std::cout << std::format("{} ", value);
19    }
20
21    std::cout << '\n';
22 }
```

2 4 6 8 10

Calculating array Element Values and an Intro to constexpr (Cont.)

- Line 13 generates values by multiplying each successive value of the loop counter, *i*, by 2 and adding 2.

```
1 // Set array values to the even integers from 2 to 10.
2 #include <format>
3 #include <iostream>
4 #include <array>
5
6 int main() {
7     // constant can be used to specify array size
8     constexpr size_t arraySize{5}; // must initialize in declaration
9
10    std::array<int, arraySize> values{}; // array values has 5 elements
11
12    for (int i{0}; i < values.size(); ++i) { // set the values
13        values.at(i) = 2 + 2 * i;
14    }
15
16    // output contents of array values in tabular format
17    for (const int& value : values) {
18        std::cout << std::format("{} ", value);
19    }
20
21    std::cout << '\n';
22 }
```

2 4 6 8 10

Calculating array Element Values and an Intro to constexpr (Cont.)

- Constants

- In Chapter 5, we used the `const` qualifier to indicate that a variable's value does not change after it's initialized.
- Sometimes, compilers will optimize your code for better performance by initializing `const` variables at compile time; however, that's not guaranteed.
- The `constexpr` qualifier also can be used to declare a constant variable.
 - ◆ Such a variable is implicitly `const` and is **explicitly initialized at compile-time**, eliminating the possibility of runtime initialization.
- Defining an array's size as a constant instead of a literal value makes programs clearer and easier to update.
 - ◆ This technique eliminates magic numbers—literal numeric values with no context to help you understand their meaning.
 - ◆ Using a constant allows you to provide a name for a literal value and can help explain that value's purpose in the program.

Calculating array Element Values and an Intro to constexpr (Cont.)

- Cannot Modify a constexpr Variable After It's Initialized
 - Line 8 uses constexpr to declare the constant arraySize with the value 5.
 - Attempting to modify arraySize after it's initialized, as in `arraySize = 7;` results in a **compilation error**.

```
1 // Set array values to the even integers from 2 to 10.
2 #include <format>
3 #include <iostream>
4 #include <array>
5
6 int main() {
7     // constant can be used to specify array size
8     constexpr size_t arraySize{5}; // must initialize in declaration
```

Calculating array Element Values and an Intro to constexpr.

- Modern C++: Doing More at Compile-Time

- A modern C++ theme is to **do more at compile-time** for better runtime performance.
 - ◆ constexpr is one of several features we'll present that eliminate runtime overhead and improve performance.
 - ◆ Each C++ standard since C++11 has expanded constexpr's use.
- In Chapter 15 you'll see that constexpr can be applied to functions.
 - ◆ When you call such a function with constexpr arguments, the compiler can determine the call's final result and replace the call with a constant, eliminating the runtime function-call overhead and thus improving performance.
 - ◆ Many functions throughout the C++ standard library are now declared constexpr to take advantage of this optimization.
- Going forward, we'll define each variable whose value can be determined at compile-time as constexpr.

Totaling Elements

- Often, array elements represent a series of values for use in a calculation.
- For example, if the elements of an array represent exam grades, a professor may wish to total the array elements and then calculate the class average for the exam.
- Processing a collection of values into a single value is known as **reduction**—a common **functional-style programming** operation.

Totaling Elements.

- The next program uses a range-based for statement (lines 11–13) to total the values in a four-element array of ints.

```
1 // Compute the sum of an array's elements.
2 #include <format>
3 #include <iostream>
4 #include <array>
5
6 int main() {
7     std::array items{10, 20, 30, 40}; // type inferred as array<int, 4>
8     int total{0};
9
10    // sum the contents of items
11    for (const int& item : items) {
12        total += item;
13    }
14
15    std::cout << std::format("Total of array elements: {}\n", total);
16 }
```

Total of array elements: 100

Using a Primitive Bar Chart to Display array Data Graphically

- Many programs present data graphically.
- For example, numeric values are often visualized in a bar chart, with longer bars representing proportionally larger values.
- One way to display numeric data graphically is with a bar chart that shows each numeric value as a bar of asterisks (*).

Using a Primitive Bar Chart to Display array Data Graphically (Cont.)

- A professor might graph the number of exam grades in several categories to visualize the grade distribution.
 - Suppose the grades were 87, 68, 94, 100, 83, 78, 85, 91, 76 and 87.
 - There was one grade of 100, two grades in the 90s, four in the 80s, two in the 70s, one in the 60s and none below 60.
 - The next program stores this data in an array of 11 elements, each corresponding to a grade range.
 - For example, element 0 contains the number of grades in the range 0–9, element 7 indicates the number of grades in the range 70–79, and element 10 indicates the number of grades of 100.
 - Upcoming examples will calculate grade frequencies based on a set of values. In this example, we initialize the array frequencies with frequency values.

Using a Primitive Bar Chart to Display array Data Graphically (Cont.)

- The program reads the numbers from frequencies and graphs them as a bar chart, displaying each grade range followed by a bar of asterisks indicating the number of grades in that range.
- Line 7 declares the array with `constexpr` because the program never modifies frequencies, and its values are known at compile-time.

```
1 // Printing a student grade distribution as a primitive bar chart.
2 #include <format>
3 #include <iostream>
4 #include <array>
5
6 int main() {
7     constexpr std::array frequencies{0, 0, 0, 0, 0, 0, 1, 2, 4, 2, 1};
```


Using a Primitive Bar Chart to Display array Data Graphically (Cont.)

- To label each bar, lines 14–20 output a grade range (e.g., "70-79: ") based on the current value of i.

```
11 // for each element of frequencies, output a bar of the chart
12 for (int i{0}; const int& frequency : frequencies) {
13     // output bar labels ("00-09:", ..., "90-99:", "100:")
14     if (i < 10) {
15         std::cout << std::format("{:02d}-{:02d}: ",
16                                   i * 10, (i * 10) + 9);
17     }
18     else {
19         std::cout << std::format("{:>5d}: ", 100);
20     }
21     ++i;
22
23     // print bar of asterisks
24     for (int stars{0}; stars < frequency; ++stars) {
25         std::cout << '*';
26     }
27
28     std::cout << '\n'; // start a new line of output
29 }
30
31 }
```

Grade distribution:

00-09:	
10-19:	
20-29:	
30-39:	
40-49:	
50-59:	
60-69:	*
70-79:	**
80-89:	****
90-99:	**
100:	*

Using a Primitive Bar Chart to Display array Data Graphically (Cont.)

- In lines 15–16, the format specifier `:02d` indicates that an integer (represented by `d`) should be formatted using a field width of 2 and a leading 0 if the integer is fewer than two digits.
- In line 19, the format specifier `:>5d` indicates that an integer should be right-aligned (`>`) in a field width of 5.

```
11 // for each element of frequencies, output a bar of the chart
12 for (int i{0}; const int& frequency : frequencies) {
13     // output bar labels ("00-09:", ..., "90-99:", "100:")
14     if (i < 10) {
15         std::cout << std::format("{:02d}-{:02d}: ",
16                                 i * 10, (i * 10) + 9);
17     }
18     else {
19         std::cout << std::format("{:>5d}: ", 100);
20     }
```

Using a Primitive Bar Chart to Display array Data Graphically.

- The nested for statement (lines 25–27) outputs the current bar's asterisks.
 - The loop-continuation condition in line 25 (stars < frequency) enables the inner for loop to count from 0 up to frequency.
 - ◆ Thus, a value from frequencies determines the number of asterisks to display.
 - In this example, frequencies elements 0–5 contain zeros because no students received a grade below 60.
 - ◆ Thus, the program displays no asterisks next to the first six grade ranges.

```
24
25 // print bar of asterisks
26 for (int stars{0}; stars < frequency; ++stars) {
27     std::cout << '*';
28 }
29 std::cout << '\n'; // start a new line of output
30 }
31 }
```

```
Grade distribution:
00-09:
10-19:
20-29:
30-39:
40-49:
50-59:
60-69: *
70-79: **
80-89: ****
90-99: **
100: *
```

Using array Elements as Counters

- Sometimes, programs use counter variables to summarize data, such as a survey's results.
- Previously the die-rolling simulation used separate counters to track the frequencies of each die face as the program rolled the die 60,000,000 times.
- We reimplement that simulation using an array of frequency counters.

```
1 // Die-rolling program using an array instead of switch.
2 #include <format>
3 #include <iostream>
4 #include <array>
5 #include <random>
6
7 int main() {
8     // set up random-number generation
9     std::random_device rd; // used to seed the default random engine
10    std::default_random_engine engine{rd()}; // rd() produces a seed
11    std::uniform_int_distribution randomDie{1, 6};
```

Using array Elements as Counters (Cont.)

- The program uses the array frequency (line 14) to count the occurrences of die values.
- Line 18 replaces the switch statement.
 - It uses a random die value as an index to determine which frequency element to increment for each die roll.
 - The dot (.) operator in line 18 has higher precedence than the ++ operator, so the statement selects a frequency element, then increments its value.
 - The call randomInt(engine) produces a random index from 1 to 6, so frequency must be large enough to store six counters.

```
13      constexpr size_t arraySize{7}; // ignore element zero
14      std::array<int, arraySize> frequency{}; // initialize to 0s
15
16      // roll die 60,000,000 times; use die value as frequency index
17      for (int roll{1}; roll <= 60'000'000; ++roll) {
18          ++frequency.at(randomDie(engine));
19      }
```

Using array Elements as Counters.

- We use a seven-element array in which we ignore element 0—it's clearer to have the die value 1 increment `frequency.at(1)` rather than `frequency.at(0)`.
 - So, each face value is used directly as a frequency index.
 - We used a counter-controlled for statement here rather than a range-based for statement, so we could skip printing element 0 of frequency.

```
21 std::cout << std::format("{}{:>13}\n", "Face", "Frequency");  
22  
23 // output each array element's value  
24 for (size_t face{1}; face < frequency.size(); ++face) {  
25     std::cout << std::format("{}{:>4}{{:>13}\n", face, frequency.at(face));  
26 }  
27 }
```

Face	Frequency
1	9997901
2	9999110
3	10001172
4	10003619
5	9997606
6	10000592

Using arrays to Summarize Survey Results

- Our next example uses arrays to summarize data collected in a survey.
- Consider the following problem statement:
Twenty students were asked to rate the food quality in the student cafeteria on a scale of 1 to 5, with 1 being “awful” and 5 being “excellent.” Place the 20 responses in an array of int values and determine the frequency of each rating.
- This is a popular type of array-processing application.
- We wish to summarize the number of responses of each rating (that is, 1–5).

Using arrays to Summarize Survey Results (Cont.)

- The array `responses` (lines 8–9) is a 20-element array of ints initialized with the students' survey responses.
 - It's declared `constexpr` because its values do not (and should not) change and are known at compile-time.
- We use a six-element array `frequency` (line 13) to count each response's number of occurrences.
 - Each frequency element is used as a counter for one of the survey responses and is initialized to zero.

```
1 // Poll analysis program.
2 #include <format>
3 #include <iostream>
4 #include <array>
5
6 int main() {
7     // place survey responses in array responses
8     constexpr std::array responses{
9         1, 2, 5, 4, 3, 5, 2, 1, 3, 1, 4, 3, 3, 3, 2, 3, 3, 2, 2, 5};
10
11     // initialize frequency counters to 0
12     constexpr size_t frequencySize{6}; // size of array frequency
13     std::array<int, frequencySize> frequency{};
```


Using arrays to Summarize Survey

```
1 // Poll analysis program.
2 #include <format>
3 #include <iostream>
4 #include <array>
5
6 int main() {
7     // place survey responses in array responses
8     constexpr std::array responses{
9         1, 2, 5, 4, 3, 5, 2, 1, 3, 1, 4, 3, 3, 3, 2, 3, 3, 2, 2, 5};
10
11     // initialize frequency counters to 0
12     constexpr size_t frequencySize{6}; // size of array frequency
13     std::array<int, frequencySize> frequency{};
14
15     // for each response in responses, use that value
16     // as frequency index to determine element to increment
17     for (const int& response : responses) {
18         ++frequency.at(response);
19     }
20
21     std::cout << std::format("{}{:>12}\n", "Rating", "Frequency");
22
23     // output each array element's value
24     for (size_t rating{1}; rating < frequency.size(); ++rating) {
25         std::cout << std::format("{}{:>6}{{:>12}\n",
26             rating, frequency.at(rating));
27     }
28 }
```

Rating	Frequency
1	3
2	5
3	7
4	2
5	3

Using arrays to Summarize Survey Results (Cont.)

- The first for statement (lines 17–19) takes one response at a time from responses (each is a value in the range 1–5) and increments one of the counters frequency.at(1) to frequency.at(5).
- The key statement is line 18, which increments the appropriate counter depending on the value of responses.at(response).

```
1 // Poll analysis program.
2 #include <format>
3 #include <iostream>
4 #include <array>
5
6 int main() {
7     // place survey responses in array responses
8     constexpr std::array responses{
9         1, 2, 5, 4, 3, 5, 2, 1, 3, 1, 4, 3, 3, 3, 2, 3, 3, 2, 2, 5};
10
11     // initialize frequency counters to 0
12     constexpr size_t frequencySize{6}; // size of array frequency
13     std::array<int, frequencySize> frequency{};
```

Using arrays to Summarize Survey Results.

- Regardless of the number of responses processed in the survey, the program requires only a six-element array (ignoring element zero) to summarize the results because all the response values are between 1 and 5, and the index values for a six-element array are 0 through 5.

```
21      std::cout << std::format("{}{:>12}\n", "Rating", "Frequency");
22
23      // output each array element's value
24      for (size_t rating{1}; rating < frequency.size(); ++rating) {
25          std::cout << std::format("{}{:>6}{{:>12}\n",
26              rating, frequency.at(rating));
27      }
28  }
```

Rating	Frequency
1	3
2	5
3	7
4	2
5	3

Sorting and Searching arrays

- **Sorting data** into ascending or descending order is one of the most important computing applications.
 - Virtually every organization must sort some data and, in many cases, massive amounts of it.
 - Sorting is an intriguing problem that has attracted some of the most intense research efforts in computer science.
- Often it may be necessary to determine whether an array contains a value that **matches a particular key** value.
 - The process of finding a key value is called **searching**.
- Let's use the built-in C++ standard library **sort function** to arrange the elements in an array into ascending order and the built-in **binary_search function** to determine whether a value is in a sorted array.

Sorting and Searching arrays (Cont.)

- Demonstrating Functions `sort` and `binary_search`
 - The next program creates an unsorted array of seven string objects named `colors` (lines 12–13).
 - The `using` declaration in line 9 enables us to initialize this array using string object literals.

```
1 // Sorting and searching arrays.
2 #include <array>
3 #include <algorithm> // contains sort and binary_search
4 #include <format>
5 #include <iostream>
6 #include <string>
7
8 int main() {
9     using namespace std::string_literals; // enables string object literals
10
11     // colors is inferred to be an array<string, 7>
12     std::array colors{"red"s, "orange"s, "yellow"s,
13                      "green"s, "blue"s, "indigo"s, "violet"s};
```

Sorting and Searching arrays (Cont.)

- Demonstrating Functions `sort` and `binary_search`
 - You form a string object literal by placing an `s` after the string's closing quote.
 - ◆ For example, in `"red"s`, the `s` following the literal tells the compiler that this literal is a `std::string` object.
 - ◆ Using string object literals here enables the compiler to infer from the initializers that the array's element type is `std::string`.

```
1 // Sorting and searching arrays.
2 #include <array>
3 #include <algorithm> // contains sort and binary_search
4 #include <format>
5 #include <iostream>
6 #include <string>
7
8 int main() {
9     using namespace std::string_literals; // enables string object literals
10
11     // colors is inferred to be an array<string, 7>
12     std::array colors{"red"s, "orange"s, "yellow"s,
13                       "green"s, "blue"s, "indigo"s, "violet"s};
```

Sorting and Searching arrays (Cont.)

- Demonstrating Functions `sort` and `binary_search`
 - Lines 17–19 display `color`'s contents. Note that the loop's control variable is declared as `const std::string&`
 - ◆ Using a reference for the range variable `color` prevents the loop from copying the current string object into `color` during each iteration.
 - ◆ In programs processing large numbers of strings, that copy operation would degrade performance.

```
15 // output original array
16 std::cout << "Unsorted colors array:\n ";
17 for (const std::string& color : colors) {
18     std::cout << std::format("{} ", color);
19 }
```

Unsorted colors array:
red orange yellow green blue indigo violet

Sorting and Searching arrays (Cont.)

- The C++ standard library function `sort`
 - Next, line 22 uses C++ standard library function `sort` (from header `<algorithm>`) to place the elements of the array `colors` into ascending order.
 - For strings, this is a lexicographical sort—that is, the strings are ordered by their characters' numerical values in the underlying character set. The `sort` function's arguments specify the range of elements to sort—in this case, the entire array.

```
21 // sort contents of colors
22 std::sort(std::begin(colors), std::end(colors));
23
24 // output sorted array
25 std::cout << "\nSorted colors array:\n ";
26 for (const std::string& color : colors) {
27     std::cout << std::format("{} ", color);
28 }
```

Sorted colors array:
blue green indigo orange red violet yellow

Sorting and Searching arrays (Cont.)

- The C++ standard library function `sort`
 - The arguments `std::begin(colors)` and `std::end(colors)` return “iterators” that represent the array’s beginning and end, respectively.
 - ◆ Chapter 13 discusses iterators in depth.
 - ◆ Functions `begin` and `end` are defined in the `<array>` header.
 - ◆ As you’ll see in later chapters, `sort` can sort the elements of several kinds of data structures.

```
21 // sort contents of colors
22 std::sort(std::begin(colors), std::end(colors));
23
24 // output sorted array
25 std::cout << "\nSorted colors array:\n ";
26 for (const std::string& color : colors) {
27     std::cout << std::format("{} ", color);
28 }
```

Sorted colors array:
blue green indigo orange red violet yellow

Sorting and Searching arrays.

- The C++ standard library function `binary_search`
 - Lines 31–32 and 37–38 use C++ standard library function `binary_search` (from header `<algorithm>`) to determine whether a value is in the array.

```
30 // search for "indigo" in colors
31 bool found{std::binary_search(
32     std::begin(colors), std::end(colors), "indigo")};
33 std::cout << std::format("\n\n\"indigo\" {} found in colors array\n",
34     found ? "was" : "was not");
35
36 // search for "cyan" in colors
37 found = std::binary_search(
38     std::begin(colors), std::end(colors), "cyan");
39 std::cout << std::format("\n\n\"cyan\" {} found in colors array\n",
40     found ? "was" : "was not");
41 }
```

"indigo" was found in colors array
"cyan" was not found in colors array

Sorting and Searching arrays.

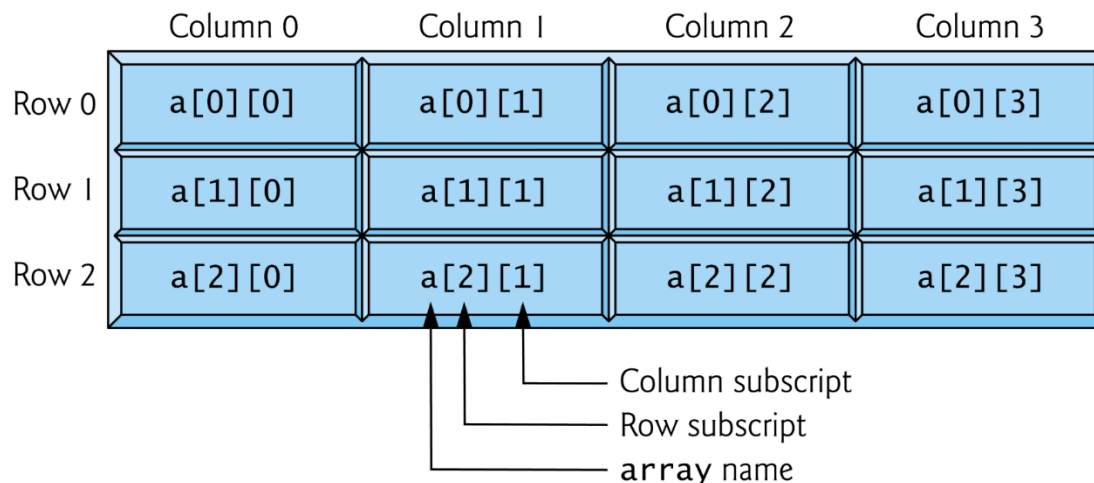
- The C++ standard library function `binary_search`
 - The sequence of values **must first be sorted in ascending order**—`binary_search` does **NOT** verify this for you.
 - ◆ Performing a binary search on an unsorted array is a logic error that could lead to incorrect results.
 - ◆ The function's first two arguments represent the range of elements to search, and the third is the search key—the value to find in the array.
 - ◆ The function returns a `bool` indicating whether the value was found.
 - ◆ In Chapter 14, we'll use the C++ Standard function `find` to obtain a search key's index in an array.

Multidimensional arrays

- You can use arrays with two dimensions (i.e., indices) to represent **tables of values** with data arranged in **rows** and **columns**.
- To identify a particular table element, we must specify two indices.
 - By convention, the first identifies the row, and the second identifies the column.
- arrays that require two indices to identify a particular element are called **two-dimensional arrays or 2-D arrays**.
- arrays with two or more dimensions are known as **multidimensional arrays**.

Multidimensional arrays (Cont.)

- The following diagram illustrates a two-dimensional array named `a`.
 - The array contains three rows and four columns, so it's said to be a **3-by-4 array**.
 - In general, an array with `m` rows and `n` columns is called an `m-by-n` array.



Multidimensional arrays (Cont.)

- We have identified every element in the diagram above with an element name of the form `a[row][column]`.
- Similarly, you can access each element with `at`, as in `a.at(i).at(j)`
- The elements names in row 0 all have a first index of 0; the elements names in column 3 all have a second index of 3.

Multidimensional arrays (Cont.)

- The next program demonstrates initializing two-dimensional arrays in declarations.
 - Lines 10–11 each create an array of arrays with two rows and three columns.

```
1 // Initializing multidimensional arrays.
2 #include <iostream>
3 #include <array>
4
5 constexpr size_t rows{2};
6 constexpr size_t columns{3};
7 void printArray(const std::array<std::array<int, columns>, rows>& a);
8
9 int main() {
10     constexpr std::array values1{std::array{1, 2, 3}, std::array{4, 5, 6}};
11     constexpr std::array values2{std::array{1, 2, 3}, std::array{4, 5, 0}};
12
13     std::cout << "values1 by row:\n";
14     printArray(values1);
15
16     std::cout << "\nvalues2 by row:\n";
17     printArray(values2);
18 }
```

values1 by row:

1 2 3

4 5 6

values2 by row:

1 2 3

4 5 0

Multidimensional arrays (Cont.)

- Declaring an array of arrays

- In lines 10–11, the compiler infers that values1 and values2 are arrays of arrays with two rows of three columns each and in which the elements are type int.
- Consider the two initializers for values1:
 - std::array{1, 2, 3}
 - std::array{4, 5, 6}
 - ◆ From these, the compiler infers that values1 has two rows.
 - ◆ Each of these initializers creates an array of three elements, so the compiler infers each row of values1 has three columns.
 - ◆ Finally, the values in these row initializers are ints, so the compiler infers that values1's element type is int.

```
10 constexpr std::array values1{std::array{1, 2, 3}, std::array{4, 5, 6}};  
11 constexpr std::array values2{std::array{1, 2, 3}, std::array{4, 5, 0}};
```


Multidimensional arrays (Cont.)

- Displaying an array of arrays
 - The program calls the function `printArray` to output each array's elements.
 - The function prototype (line 7) and definition (lines 21–31) specify that `printArray` receives a two-row and three-column array of ints via a `const` reference.

```
20 // output array with two rows and three columns
21 void printArray(const std::array<std::array<int, columns>, rows>& a) {
22     // loop through array's rows
23     for (const auto& row : a) {
24         // loop through columns of current row
25         for (const auto& element : row) {
26             std::cout << element << ' ';
27         }
28
29         std::cout << '\n'; // start new line of output
30     }
31 }
```

Multidimensional arrays (Cont.)

- Displaying an array of arrays
 - In line 21, the outer array type `const std::array<std::array<int, columns>, rows>&` indicates that it has rows (2) elements of type `array<int, columns>`
 - So, each of the outer array's elements is an array of ints containing columns (3) elements.
 - The parameter receives the array as a const reference because `printArray` does not modify the elements.

```
20 // output array with two rows and three columns
21 void printArray(const std::array<std::array<int, columns>, rows>& a) {
22     // loop through array's rows
23     for (const auto& row : a) {
24         // loop through columns of current row
25         for (const auto& element : row) {
26             std::cout << element << ' ';
27         }
28
29         std::cout << '\n'; // start new line of output
30     }
31 }
```

Multidimensional arrays (Cont.)

- Nested Range-Based for Statements

- To process the elements of a two-dimensional array, we use a nested loop:
 - ◆ the outer loop iterates through the rows and
 - ◆ the inner loop iterates through the columns of a given row.
- The nested loop is implemented with range-based for.
- Lines 23 and 25 introduce the `auto` keyword, which tells the compiler to infer (determine) a variable's data type based on the variable's initializer value.

```
20 // output array with two rows and three columns
21 void printArray(const std::array<std::array<int, columns>, rows>& a) {
22     // loop through array's rows
23     for (const auto& row : a) {
24         // loop through columns of current row
25         for (const auto& element : row) {
26             std::cout << element << ' ';
27         }
28
29         std::cout << '\n'; // start new line of output
30     }
31 }
```

Multidimensional arrays (Cont.)

- Nested Range-Based for Statements

- The outer loop's range variable `row` is initialized with an element from the parameter `a`.
 - ◆ Looking at the array's declaration, you can see that it contains elements of type `array<int, columns>` so the compiler infers that `row` refers to a three-element array of `int` values (again, `columns` is 3).
 - ◆ The `const&` in `row`'s declaration indicates that the reference cannot be used to modify the rows and prevents each row from being copied into the range variable.

```
20 // output array with two rows and three columns
21 void printArray(const std::array<std::array<int, columns>, rows>& a) {
22     // loop through array's rows
23     for (const auto& row : a) {
24         // loop through columns of current row
25         for (const auto& element : row) {
26             std::cout << element << ' ';
27         }
28
29         std::cout << '\n'; // start new line of output
30     }
31 }
```

Multidimensional arrays (Cont.)

- Nested Range-Based for Statements
 - The inner loop's range variable `element` is initialized with one element of the array represented by `row`.
 - ◆ So, the compiler infers that `element` is a reference to a `const int` because each row contains three `int` values.

```
20 // output array with two rows and three columns
21 void printArray(const std::array<std::array<int, columns>, rows>& a) {
22     // loop through array's rows
23     for (const auto& row : a) {
24         // loop through columns of current row
25         for (const auto& element : row) {
26             std::cout << element << ' ';
27         }
28
29         std::cout << '\n'; // start new line of output
30     }
31 }
```

Multidimensional arrays (Cont.)

- Nested Counter-Controlled for Statements

- We could have implemented the nested loop with counter-controlled iteration as follows:

```
for (size_t row{0}; row < a.size(); ++row) {  
    for (size_t column{0}; column < a.at(row).size(); ++column) {  
        cout << a.at(row).at(column) << ' '; // or a[row][column]  
    }  
    cout << '\n';  
}
```

```
20 // output array with two rows and three columns  
21 void printArray(const std::array<std::array<int, columns>, rows>& a) {  
22     // loop through array's rows  
23     for (const auto& row : a) {  
24         // loop through columns of current row  
25         for (const auto& element : row) {  
26             std::cout << element << ' ';  
27         }  
28         std::cout << '\n'; // start new line of output  
29     }  
30 }  
31
```

Multidimensional arrays (Cont.)

- Initializing an array of arrays with a Fully Braced Initializer List
 - If you specify the array's dimensions explicitly when you declare it, you can simplify its initializer list.
 - For example, line 10 could be written as

```
constexpr std::array<std::array<int, columns>, rows> values1{
    {{1, 2, 3}, // row 0
     {4, 5, 6}} // row 1
};
```
 - In this case, if an initializer sublist has fewer elements than the number of columns, the row's remaining elements would be value initialized.

Multidimensional arrays (Cont.)

- Common Two-Dimensional array Manipulations: Setting a Row's Values
 - Let's consider several other common manipulations using the three-row and four-column array a.
 - The following for statement sets all the elements in row 2 to zero:

```
for (size_t column{0}; column < a.at(2).size(); ++column) {  
    a.at(2).at(column) = 0; // or a[2][column]  
}
```

 - ◆ The for statement varies only the second index (i.e., the column index).
 - The preceding for statement is equivalent to the following assignment statements:
 - ◆ `a.at(2).at(0) = 0; // or a[2][0] = 0;`
 - ◆ `a.at(2).at(1) = 0; // or a[2][1] = 0;`
 - ◆ `a.at(2).at(2) = 0; // or a[2][2] = 0;`
 - ◆ `a.at(2).at(3) = 0; // or a[2][3] = 0;`

Multidimensional arrays (Cont.)

- Common Two-Dimensional array Manipulations: Totaling All the Elements with Nested Counter-Controlled for Loops

- The following nested counter-controlled for statement totals the elements in the array a:

```
int total{0};  
for (size_t row{0}; row < a.size(); ++row) {  
    for (size_t column{0}; column < a.at(row).size(); ++column) {  
        total += a.at(row).at(column); // a[row][column]  
    }  
}
```

- The for statement totals the array's elements one row at a time.
 - ◆ The outer loop begins by setting the row index to 0, so the elements of row 0 may be totaled by the inner loop.
 - ◆ The outer loop then increments row to 1, so the elements of row 1 can be totaled.
 - ◆ Then, the outer for statement increments row to 2, so the elements of row 2 can be totaled.

Multidimensional arrays (Cont.)

- Common Two-Dimensional array Manipulations:
Totaling All the Elements with Nested Range-Based for Loops

- Nested range-based for statements are the preferred way to implement the preceding loop:

```
int total{0};  
for (const auto& row : a) { // for each row in a  
    for (const auto& column : row) { // for each column in row  
        total += column;  
    }  
}
```

Multidimensional arrays.

- mdarray
 - The C++ Standard Committee is working on a true multidimensional array container called mdarray for future C++ versions.
 - You can follow the progress on mdarray at
 - ◆ mdarray—<https://isocpp.org/files/papers/D1684R0.html>

Intro to Functional-Style Programming.

- Like other popular languages, such as Python, Java and C#, C++ supports several programming paradigms—procedural, object-oriented, generic (template-oriented) and “functional-style.”
- C++’s “functional-style” features help you write more concise code that’s easier to read, debug and modify, and has fewer errors.
- Functional-style programs also can be easier to parallelize to get better performance on today’s multi-core processors.

What vs. How

- As a program's tasks get more complicated, the code can become harder to read, debug and modify, and more likely to contain errors. Specifying how the code works can become complex.
- With functional-style programming, you specify what you want to do, and library code typically handles “the how” for you. This can eliminate many errors.

What vs. How.

- Consider the following range-based for statement, which totals the elements of the array `integers`:

```
for (const int& item : integers) {  
    total += item;  
}
```
- Though this procedural code hides the iteration details, we still have to specify how to total the elements by adding each item to the variable `total`.
 - Each time you modify a variable, you can introduce errors.
- Functional-style programming emphasizes **immutability**—it avoids operations that modify variables' values.
- If this is your first exposure to functional-style programming, you might be wondering, “How can this be?” Read on.

Functional-Style Reduction with `accumulate`

- The next program replaces range-based for statement with a call to the C++ standard library `accumulate` algorithm (from header `<numeric>`).
 - By default, this function knows how to compute the sum of a range's values, reducing them to a single value.
 - This is known as a `reduction` and is a common functional-style programming operation.

```
1 // Compute the sum of the elements of an array using accumulate.
2 #include <array>
3 #include <format>
4 #include <iostream>
5 #include <numeric>
6
7 int main() {
8     constexpr std::array integers{10, 20, 30, 40};
9     std::cout << std::format("Total of array elements: {}\n",
10                             std::accumulate(std::begin(integers), std::end(integers), 0));
11 }
```

Total of array elements: 100

Functional-Style Reduction with accumulate (Cont.)

- Like function sort, function accumulate's first two arguments (line 10) specify the range of elements to sum—in this case, the elements from the beginning to the end of integers.
 - The function internally adds to the running total of the elements it processes, hiding those calculations.
 - The third argument is the running total's initial value (0), and this argument's type is accumulate's return type.

```
7  int main() {  
8      constexpr std::array integers{10, 20, 30, 40};  
9      std::cout << std::format("Total of array elements: {}\\n",  
10         std::accumulate(std::begin(integers), std::end(integers), 0));  
11 }
```


Functional-Style Reduction with `accumulate`.

- Function `accumulate` uses internal iteration, which also is hidden from you.
 - The function knows how to iterate through a range of elements and add each element to the running total.
 - Stating what you want to do and letting the library determine how to do it is known as **declarative programming**—another hallmark of functional programming.

```
7 int main() {  
8     constexpr std::array integers{10, 20, 30, 40};  
9     std::cout << std::format("Total of array elements: {}\n",  
10         std::accumulate(std::begin(integers), std::end(integers), 0));  
11 }
```

Passing Functions as Arguments to Other Functions: Introducing Lambda Expressions

- Many standard library functions allow you to customize how they work by passing other functions as arguments.
- Functions that receive other functions as arguments are called **higher-order functions** and are commonly used in functional programming.
- Consider function `accumulate`, which totals elements by default. It also provides an overload, which receives as its **fourth argument a function** that defines **how to perform the reduction**.
- Rather than simply totaling the values, the next program calculates the product of the values.

Passing Functions as Arguments to Other Functions: Introducing Lambda Expressions (Cont.)

```
1 // Compute the product of an array's elements using accumulate.
2 #include <array>
3 #include <format>
4 #include <iostream>
5 #include <numeric>
6
7 int multiply(int x, int y) {
8     return x * y;
9 }
10
11 int main() {
12     constexpr std::array integers{1, 2, 3, 4, 5};
13
14     std::cout << std::format("Product of integers: {}\n", std::accumulate(
15         std::begin(integers), std::end(integers), 1, multiply));
16
17     std::cout << std::format("Product of integers with a lambda: {}\n",
18         std::accumulate(std::begin(integers), std::end(integers), 1,
19             [](const auto& x, const auto& y) {return x * y; }));
20 }
```

Product of integers: 120
Product of integers with a lambda: 120

Passing Functions as Arguments to Other Functions: Introducing Lambda Expressions (Cont.)

- Calling accumulate with a Named Function

- Lines 14–15 call accumulate for the array integers (defined at line 12).
 - ◆ We're calculating the product, so the third argument (i.e., the initial value of the reduction) is 1 rather than 0; otherwise, the final product would be 0.

```
1 // Compute the product of an array's elements using accumulate.
2 #include <array>
3 #include <format>
4 #include <iostream>
5 #include <numeric>
6
7 int multiply(int x, int y) {
8     return x * y;
9 }
10
11 int main() {
12     constexpr std::array integers{1, 2, 3, 4, 5};
13
14     std::cout << std::format("Product of integers: {}\n", std::accumulate(
15         std::begin(integers), std::end(integers), 1, multiply));
```

Product of integers: 120

Passing Functions as Arguments to Other Functions: Introducing Lambda Expressions (Cont.)

- Calling accumulate with a Named Function
 - The fourth argument is the function to call for every array element—in this case, multiply (defined in lines 7–9). To calculate a product, this function must receive two arguments:
 - ◆ the product so far and
 - ◆ a value from the array
 - and must return a value, which becomes the new product.
 - As accumulate iterates through integers, it passes the current product and the next element as arguments.

```
7 int multiply(int x, int y) {  
8     return x * y;  
9 }  
10  
11 int main() {  
12     constexpr std::array integers{1, 2, 3, 4, 5};  
13  
14     std::cout << std::format("Product of integers: {}\n", std::accumulate(  
15         std::begin(integers), std::end(integers), 1, multiply));  
}
```

Passing Functions as Arguments to Other Functions: Introducing Lambda Expressions (Cont.)

- Calling accumulate with a Named Function
 - For this example, accumulate internally calls multiply five times:
 - ◆ The first call passes the initial product (1, specified as accumulate's third argument) and the array's first element (1), producing the product 1.
 - ◆ The second call passes the current product (1) and the array's second element (2), producing the product 2.
 - ◆ The third call passes 2 and the array's third element (3), producing the product 6.
 - ◆ The fourth call passes 6 and the array's fourth element (4), producing the product 24.
 - ◆ Finally, the last call passes 24 and the array's fifth element (5), producing the overall result 120, which accumulate returns to its caller.

Passing Functions as Arguments to Other Functions: Introducing Lambda Expressions (Cont.)

- Calling accumulate with a Lambda Expression
 - Sometimes, you do not need to reuse a function, in which case, you can define a function where it's needed by using a **lambda expression** (or simply **lambda**).
 - ◆ A lambda expression is an **anonymous function**—that is, a function without a name.
 - The call to accumulate in lines 18–19 uses the following lambda expression to perform the same task as multiply:
`[](const auto& x, const auto& y){return x * y;}`
 - Lambdas begin with the **lambda introducer** (`[]`), followed by a **comma-separated parameter list** and a **function body**.
 - This lambda receives two parameters, calculates their product and returns the result.

Product of integers with a lambda: 120

```
17  std::cout << std::format("Product of integers with a lambda: {}\n",
18      std::accumulate(std::begin(integers), std::end(integers), 1,
19          [](const auto& x, const auto& y) {return x * y; }));
20  }
```

Passing Functions as Arguments to Other Functions: Introducing Lambda Expressions (Cont.)

- Calling accumulate with a Lambda Expression
 - You saw that auto enables the compiler to infer a variable's type based on its initial value.
 - Specifying a lambda parameter's type as auto enables the compiler to infer the type based on the context in which the lambda is called.
 - In this example, accumulate calls the lambda once for each array element, passing the current product and the element's value as the lambda's arguments.
 - Since the initial product (1) is an int and the array contains ints, the compiler infers the lambda parameters' types as int.
 - The preceding lambda is a **generic lambda**—it uses auto to infer each parameter's type.
 - The compiler also infers the lambda's return type from the expression $x * y$ —both x and y are ints, so this lambda returns an int.

Passing Functions as Arguments to Other Functions: Introducing Lambda Expressions.

- Calling accumulate with a Lambda Expression
 - We declared the parameters as const references:
 - ◆ They are const, so the lambda's body cannot modify the caller's variables.
 - ◆ They are references for performance to ensure that if the lambda is used with large objects, it does not copy them.
 - This lambda could be used with any type that supports the * operator.
 - Chapter 14 discusses lambda expressions in detail.

Filter, Map and Reduce: Intro to C++20's Ranges Library

- The C++ standard library has enabled functional-style programming for many years.
- C++20's new ranges library (header `<ranges>`) makes functional-style programming more convenient.
- Here, we introduce two key aspects of this library—ranges and views:
 - A **range** is a **collection of elements** that you can **iterate** over. So an array, for example, is a range.
 - A **view** enables you to **specify an operation** that manipulates a range. Views are composable—you can chain them together to process a range's elements through multiple operations.

Filter, Map and Reduce: Intro to C++20's Ranges Library (Cont.)

- showValues Lambda for Displaying This Application's Results
 - The next program demonstrates several functional-style operations using C++20 ranges.
 - Throughout this example, we display various range operations' results using a range-based for statement.
 - Rather than repeating that code, we could define a function that receives a range and displays its values.

Filter, Map and Reduce: Intro to C++20's Ranges Library (Cont.)

- showValues Lambda for Displaying This Application's Results
 - Instead, we defined a generic lambda (lines 11–19) to show that you can store a lambda in a local variable (showValues).
 - Then, you can use the variable's name to call the lambda.

```
1 // Functional-style programming with C++20 ranges and views.
2 #include <array>
3 #include <format>
4 #include <iostream>
5 #include <numeric>
6 #include <ranges>
7
8 int main() {
9     // lambda to display results of range operations
10    auto showValues{
11        [](auto& values, const std::string& message) {
12            std::cout << std::format("{}: ", message);
13
14            for (const auto& value : values) {
15                std::cout << std::format("{} ", value);
16            }
17
18            std::cout << '\n';
19        }
20    };
```

Filter, Map and Reduce: Intro to C++20's Ranges Library (Cont.)

- Generating a Sequential Range of Integers with `views::iota`
 - In many of this chapter's examples, we created an array, then processed its values.
 - ◆ This required preallocating the array with the appropriate number of elements.
 - ◆ Sometimes, you can generate values **on-demand** rather than creating and storing the values in advance.
 - ◆ Operations that generate values on demand use **lazy evaluation**, which can reduce your program's memory consumption and improve performance when all the values are not needed at once.
 - ◆ Line 22 uses the `<ranges>` library's `views::iota` to generate a range of integers from its first argument (1) up to, but not including, its second argument (11).
 - This is known as a **half-open range**.
 - ◆ The values are not generated until the program iterates over the results, such as when we call `showValues` (line 23) to display the values.

```
22 auto values1{std::views::iota(1, 11)}; // generate integers 1-10
23 showValues(values1, "Generate integers 1-10");
```

Generate integers 1-10: 1 2 3 4 5 6 7 8 9 10

Filter, Map and Reduce: Intro to C++20's Ranges Library (Cont.)

- Filtering Items with `views::filter`

- A common functional-style programming operation is **filtering** elements to **select only those that match a condition**.
 - ◆ This often produces fewer elements than the range being filtered.
- One way to implement filtering would be a loop that iterates over the elements, using an if statement to check whether each element matches a condition.
 - ◆ You could then do something with that element, such as add it to a container.
 - ◆ That requires explicitly defining an iteration control statement and mutable variables, which can be error-prone.

Filter, Map and Reduce: Intro to C++20's Ranges Library (Cont.)

- Filtering Items with `views::filter`

- With ranges and views, we can use `views::filter` to focus on what we want to accomplish—in this case, getting the even integers in the range 1–10.
- The `values2` initializer in lines 26–27 uses the `|` operator to connect multiple operations.
 - ◆ The first operation (`values1` from line 22) generates 1–10, and the second filters those results.
 - ◆ Together, these operations form a pipeline.
 - ◆ Each pipeline begins with a range, which is the data source (the values 1–10 produced by `values1`), followed by an arbitrary number of operations, each separated from the next by `|`.

```
25 // filter each value in values1, keeping only the even integers
26 auto values2{values1 |
27   std::views::filter([](const auto& x) {return x % 2 == 0;})};
28 showValues(values2, "Filtering even integers");
```

Filtering even integers: 2 4 6 8 10

Filter, Map and Reduce: Intro to C++20's Ranges Library (Cont.)

- Filtering Items with `views::filter`

- The argument to `views::filter` must be a **function that receives one value to process and returns a bool** indicating whether to keep the value.
 - ◆ We passed a lambda that returns true if its argument is divisible by 2.
- After lines 26–27 execute, `values2` represents a lazy pipeline that can generate the integers 1–10 and filter those values, keeping only the even integers.
- The pipeline concisely represents what we want to do but not how to do it:
 - ◆ `views::iota` knows how to generate integers and
 - ◆ `views::filter` knows how to use its function argument to determine whether to keep each value received from earlier in the pipeline.

Filter, Map and Reduce: Intro to C++20's Ranges Library (Cont.)

- Filtering Items with `views::filter`
 - However, the pipeline is *lazy*—no results are produced until you iterate over `values2`.
 - ◆ When `showValues` iterates over `values2`, `views::iota` produces a value, then `views::filter` calls its function argument to determine whether to keep the value.
 - ◆ If that function returns `true`, the range-based for statement receives that value from the pipeline and displays it.
 - ◆ Otherwise, the processing steps repeat with the next value generated by `views::iota`.

Filter, Map and Reduce: Intro to C++20's Ranges Library (Cont.)

- Mapping Items with `views::transform`
 - Another common functional-style programming operation is **mapping** elements to new values, possibly of different types.
 - Mapping produces a result with **the same number of elements** as the original range being mapped.
 - With C++20 ranges, **`views::transform` performs mapping operations**. The pipeline in lines 31–32 adds another operation to the pipeline from lines 26–27, mapping the filtered results from `values2` to their squares with the lambda expression passed to `views::transform` in line 32.

```
30 // map each value in values2 to its square
31 auto values3{
32     values2 | std::views::transform([](const auto& x) {return x * x;});
33     showValues(values3, "Mapping even integers to squares");
```

Mapping even integers to squares: 4 16 36 64 100

Filter, Map and Reduce: Intro to C++20's Ranges Library (Cont.)

- Mapping Items with `views::transform`

- The argument to `views::transform` is a function that receives a value to process and returns the mapped value, possibly of a different type.
- When `showValues` iterates over the results of the `values3` pipeline:
 1. `views::iota` produces a value.
 2. `views::filter` determines whether the value is even.
 - If so, the value is passed to Step 3.
 - Otherwise, processing proceeds with the next value generated by `views::iota` in Step 1.
 3. `views::transform` calculates the square of the even integer (as specified by line 32's lambda), then the range-based for loop in `showValues` displays the value and processing proceeds with the next value generated by `views::iota` in Step 1.

Filter, Map and Reduce: Intro to C++20's Ranges Library (Cont.)

- Combining Filtering and Mapping Operations into a Pipeline
 - A pipeline may contain an arbitrary number of operations separated by `|` operators.
 - The pipeline in lines 36–38 combines the preceding operations into a single pipeline, and line 39 displays the results.

```
36 auto values4{  
37     values1 | std::views::filter([](const auto& x) {return x % 2 == 0;})  
38     | std::views::transform([](const auto& x) {return x * x; });  
39     showValues(values4, "Squares of even integers");
```

Squares of even integers: 4 16 36 64 100

Filter, Map and Reduce: Intro to C++20's Ranges Library (Cont.)

- Reducing a Range Pipeline with `accumulate`
 - C++ standard library functions like `accumulate` also work with lazy range pipelines.
 - Line 43 performs a reduction that sums the squares of the even integers produced by the pipeline in lines 36–38.

```
41 // total the squares of the even integers
42 std::cout << std::format("Sum squares of even integers 2-10: {}\n",
43     std::accumulate(std::begin(values4), std::end(values4), 0));
```

Sum squares of even integers 2-10: 220

Filter, Map and Reduce: Intro to C++20's Ranges Library.

- Filtering and Mapping an Existing Container's Elements
 - Various C++ containers, including arrays and vectors, can be used as the data source in a range pipeline.
 - Line 46 creates an array containing 1–10, then uses it in a pipeline that calculates the squares of the even integers in the array.

```
45 // process a container's elements
46 constexpr std::array numbers{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
47 auto values5{
48     numbers | std::views::filter([](const auto& x) {return x % 2 == 0;})
49             | std::views::transform([](const auto& x) {return x * x; });
50     showValues(values5, "Squares of even integers in array numbers");
51 }
```

Squares of even integers in array numbers: 4 16 36 64 100

Objects-Natural Case Study: C++ Standard Library Class Template vector

- We now continue our objects-natural presentation by taking objects of C++ standard library class template vector (from header <vector>).
 - A vector is similar to an array but supports dynamic resizing.
 - At the end of this section, we'll demonstrate vector's bounds-checking capabilities, which array also has.
 - ◆ There, we'll introduce C++'s exception-handling mechanism by detecting and handling an out-of-bounds vector index.
 - ◆ At that point, we'll discuss the <stdexcept> header (line 4).

```
1 // Demonstrating C++ standard library class template vector.  
2 #include <iostream>  
3 #include <vector>  
4 #include <stdexcept>
```

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Lines 6–7 are the function prototypes for functions `outputVector` (lines 92–98) and `inputVector` (lines 101–105), which display a vector's contents and input values into a vector, respectively.

```
1 // Demonstrating C++ standard library class template vector.
2 #include <iostream>
3 #include <vector>
4 #include <stdexcept>
5
6 void outputVector(const std::vector<int>& items); // display the vector
7 void inputVector(std::vector<int>& items); // input values into the vector
8
```


Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Creating vector Objects

- Lines 10–11 create two vector objects that store values of type `int`—`integers1` contains seven elements, and `integers2` contains 10 elements.
- By default, each vector object's elements are set to 0.
- Like arrays, vectors can store most data types—simply replace `int` in `std::vector<int>` with the appropriate type.

```
9  int main() {  
10     std::vector<int> integers1(7); // 7-element vector<int>  
11     std::vector<int> integers2(10); // 10-element vector<int>
```

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Creating vector Objects

- Note that we used **parentheses** rather than a braced initializer to pass the size argument to each vector object's constructor.
 - ◆ When creating a vector, if the braces contain one value of the vector's element type, the compiler treats the braces as a one-element initializer list rather than the vector's size.
 - ◆ So the following declaration actually creates a **one-element** `vector<int>` containing the value 7, **NOT** a seven-element vector:
`std::vector<int> integers1{7};`
- If you know the vector's contents at compile-time, you can use an initializer list and **class template argument deduction (CTAD)** to define the vector, as we've done with arrays.
 - ◆ For example, the following statement defines a four-element vector of doubles:
`std::vector integers1{1.1, 2.2, 3.3, 4.4};`

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- vector Member Function size; Function outputVector
 - Line 14 uses vector member function size to obtain integers1's number of elements.
 - Line 16 passes integers1 to function outputVector (lines 92–98), which displays the integers2.

```
13 // print integers1 size and contents
14 std::cout << "Size of vector integers1 is " << integers1.size()
15 << "\nvector after initialization:";
16 outputVector(integers1);
17
18 // print integers2 size and contents
19 std::cout << "\nSize of vector integers2 is " << integers2.size()
20 << "\nvector after initialization:";
21 outputVector(integers2);
```

Size of vector integers1 is 7
vector after initialization: 0 0 0 0 0 0 0

Size of vector integers2 is 10
vector after initialization: 0 0 0 0 0 0 0 0 0 0

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Function inputVector

- Lines 25–26 pass integers1 and integers2 to function inputVector (lines 101–105) to read values for each vector's elements from the user.

```
23 // input and print integers1 and integers2
24 std::cout << "\nEnter 17 integers:\n";
25 inputVector(integers1);
26 inputVector(integers2);
27
28 std::cout << "\nAfter input, the vectors contain:\n"
29 << "integers1:";
30 outputVector(integers1);
31 std::cout << "integers2:";
32 outputVector(integers2);
```

Enter 17 integers:

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

After input, the vectors contain:

integers1: 1 2 3 4 5 6 7

integers2: 8 9 10 11 12 13 14 15 16 17

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Comparing vector Objects for Inequality
 - In Chapter 5, we introduced function overloading. A similar concept is operator overloading, which allows you to define how a built-in operator works for a custom type.
 - The C++ standard library defines overloaded operators == and != that compare two arrays or two vectors for equality or inequality, respectively.

```
34 // use inequality (!=) operator with vector objects
35 std::cout << "\nEvaluating: integers1 != integers2\n";
36
37 if (integers1 != integers2) {
38     std::cout << "integers1 and integers2 are not equal\n";
39 }
```

```
Evaluating: integers1 != integers2
integers1 and integers2 are not equal
```

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Initializing One vector with the Contents of Another
 - You can initialize a new vector by copying the contents of an existing one (Line 43).
 - ◆ Here, we used CTAD to infer integers3's element type from that of integers1.
 - ◆ Line 43 invokes the class template vector's **copy constructor** to perform the copy operation.

```
41 // create vector integers3 using integers1 as an
42 // initializer; print size and contents
43 std::vector integers3{integers1}; // copy constructor
44
45 std::cout << "\nSize of vector integers3 is " << integers3.size()
46 << "\nvector after initialization: ";
47 outputVector(integers3);
```

```
Size of vector integers3 is 7
vector after initialization: 1 2 3 4 5 6 7
```

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Assigning vectors and Comparing vectors for Equality
 - Line 51 assigns integers2 to integers1, demonstrating that the assignment (=) operator is overloaded to work with vector objects.
 - Lines 53–56 output the contents of both objects to show that they now contain identical values.
 - Line 61 then compares integers1 to integers2 with the equality (==) operator to determine whether the contents of the two objects are equal after the assignment, which they are.

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Assigning vectors and Comparing vectors for Equality

```
49 // use overloaded assignment (=) operator
50 std::cout << "\nAssigning integers2 to integers1:\n";
51 integers1 = integers2; // assign integers2 to integers1
52
53 std::cout << "integers1: ";
54 outputVector(integers1);
55 std::cout << "integers2: ";
56 outputVector(integers2);
57
58 // use equality (==) operator with vector objects
59 std::cout << "\nEvaluating: integers1 == integers2\n";
60
61 if (integers1 == integers2) {
62     std::cout << "integers1 and integers2 are equal\n";
63 }
```

Assigning integers2 to integers1:
integers1: 8 9 10 11 12 13 14 15 16 17
integers2: 8 9 10 11 12 13 14 15 16 17

Evaluating: integers1 == integers2
integers1 and integers2 are equal

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Using the at Member Function to Access and Modify vector Elements
 - Lines 66 and 70 use the vector member function at to obtain an element and use it to get a value from the vector and replace a value in the vector, respectively. If the index is valid, at returns either
 - ◆ a reference to that element, which can be used to change the vector element's value, or
 - ◆ a const reference to that element, which can be used to read but not change the element's value. The at function returns a const reference if it's called on a const vector or via a const reference.

```
65 // use the value at location 5 as an rvalue
66 std::cout << "\nintegers1.at(5) is " << integers1.at(5);
67
68 // use integers1.at(5) as an lvalue
69 std::cout << "\n\nAssigning 1000 to integers1.at(5)\n";
70 integers1.at(5) = 1000;
71 std::cout << "integers1: ";
72 outputVector(integers1);
```

```
integers1.at(5) is 13
Assigning 1000 to integers1.at(5)
integers1: 8 9 10 11 12 1000 14 15 16 17
```

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Using the at Member Function to Access and Modify vector Elements
 - Like arrays, vectors have a [] operator. C++ does not require bounds checking when accessing elements with [].
 - Therefore, you must ensure that operations using [] do not accidentally manipulate elements outside the vector's bounds.

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Exception Handling: Processing an Out-of-Range Index
 - Line 77 attempts to output the value in `integers1.at(15)`, which is outside the vector's bounds.
 - Member function `at`'s bounds checking recognizes the invalid index and **throws an exception**, which indicates a problem at execution time.
 - The name “exception” suggests that the problem occurs infrequently.

```
74 // attempt to use out-of-range index
75 try {
76     std::cout << "\nAttempt to display integers1.at(15)\n";
77     std::cout << integers1.at(15) << '\n'; // ERROR: out of range
78 }
79 catch (const std::out_of_range& ex) {
80     std::cerr << "An exception occurred: " << ex.what() << '\n';
81 }
```

Attempt to display `integers1.at(15)`
An exception occurred: invalid vector subscript

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Exception Handling: Processing an Out-of-Range Index
 - Exception handling enables you to create fault-tolerant programs that can handle (or catch) exceptions.
 - ◆ Sometimes, this allows a program to continue executing as if no problems were encountered.
 - ◆ For example, this program still runs to completion, even though we attempt to access an out-of-range index.
 - ◆ More severe problems might prevent a program from continuing normal execution, requiring it to terminate after cleaning up any resources it uses (such as closing files or database connections).

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- The try Statement

- By default, an exception causes a C++ program to terminate.
- To handle an exception and possibly enable the program to continue executing, place any code that might throw an exception in a try statement (lines 75–81).

```
74 // attempt to use out-of-range index
75 try {
76     std::cout << "\nAttempt to display integers1.at(15)\n";
77     std::cout << integers1.at(15) << '\n'; // ERROR: out of range
78 }
79 catch (const std::out_of_range& ex) {
80     std::cerr << "An exception occurred: " << ex.what() << '\n';
81 }
```

Attempt to display integers1.at(15)
An exception occurred: invalid vector subscript

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- The try Statement

- The try block (lines 75–78) contains the code that might throw an exception, and
- the catch block (lines 79–81) contains the code that handles the exception if one occurs.
- The braces that delimit try and catch blocks' bodies are required.
- If the code in the try block executes successfully, the catch block is ignored.

```
74 // attempt to use out-of-range index
75 try {
76     std::cout << "\nAttempt to display integers1.at(15)\n";
77     std::cout << integers1.at(15) << '\n'; // ERROR: out of range
78 }
79 catch (const std::out_of_range& ex) {
80     std::cerr << "An exception occurred: " << ex.what() << '\n';
81 }
```

Attempt to display integers1.at(15)
An exception occurred: invalid vector subscript

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Executing the catch Block
 - When the program calls the vector member function `at` with the argument 15 (line 77), the function attempts to access the element at location 15, which is outside the vector's bounds—`integers1` has only 10 elements at this point.
 - Bounds checking is performed at execution time, so it generates an exception.

```
74 // attempt to use out-of-range index
75 try {
76     std::cout << "\nAttempt to display integers1.at(15)\n";
77     std::cout << integers1.at(15) << '\n'; // ERROR: out of range
78 }
79 catch (const std::out_of_range& ex) {
80     std::cerr << "An exception occurred: " << ex.what() << '\n';
81 }
```

Attempt to display `integers1.at(15)`
An exception occurred: invalid vector subscript

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Executing the catch Block

- Specifically, line 77 throws an `out_of_range` exception (from header `<stdexcept>`) to notify the program of this problem.
 - ◆ This immediately terminates the try block, skipping any remaining statements in that block.
 - ◆ Then, the catch block begins executing. If you declared any variables in the try block, they're now out of scope and not accessible to the catch block.

```
74 // attempt to use out-of-range index
75 try {
76     std::cout << "\nAttempt to display integers1.at(15)\n";
77     std::cout << integers1.at(15) << '\n'; // ERROR: out of range
78 }
79 catch (const std::out_of_range& ex) {
80     std::cerr << "An exception occurred: " << ex.what() << '\n';
81 }
```

Attempt to display integers1.at(15)
An exception occurred: invalid vector subscript

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Executing the catch Block
 - A catch block should declare its exception parameter (ex) as a const reference.
 - The catch block can handle exceptions of the specified type.
 - ◆ Inside the block, you can use the parameter's identifier to interact with a caught exception object.

```
74 // attempt to use out-of-range index
75 try {
76     std::cout << "\nAttempt to display integers1.at(15)\n";
77     std::cout << integers1.at(15) << '\n'; // ERROR: out of range
78 }
79 catch (const std::out_of_range& ex) {
80     std::cerr << "An exception occurred: " << ex.what() << '\n';
81 }
```

Attempt to display integers1.at(15)
An exception occurred: invalid vector subscript

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- what Member Function of the Exception Object
 - When lines 79–81 catch the exception, the program displays a message (which varies by compiler) indicating the problem that occurred.
 - Line 80 calls the exception object's `what` member function to get the error message.
 - Once the message is displayed in this example, the exception is considered handled, and the program continues with the next statement after the catch block's closing brace.

```
74 // attempt to use out-of-range index
75 try {
76     std::cout << "\nAttempt to display integers1.at(15)\n";
77     std::cout << integers1.at(15) << '\n'; // ERROR: out of range
78 }
79 catch (const std::out_of_range& ex) {
80     std::cerr << "An exception occurred: " << ex.what() << '\n';
81 }
```

Attempt to display integers1.at(15)
An exception occurred: invalid vector subscript

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Changing the Size of a vector

- One key difference between a vector and an array is that a vector can dynamically grow and shrink as its number of elements varies.
- To demonstrate this, line 84 shows the current size of integers3, line 85 calls the vector's push_back member function to add a new element containing 1000 to the end of the vector, and line 86 shows the new size of integers3. Line 88 then displays integers3's new contents.

```
83 // changing the size of a vector
84 std::cout << "\nCurrent integers3 size is: " << integers3.size();
85 integers3.push_back(1000); // add 1000 to the end of the vector
86 std::cout << "\nNew integers3 size is: " << integers3.size()
87 << "\nintegers3 now contains: ";
88 outputVector(integers3);
89 }
```

```
Current integers3 size is: 7
New integers3 size is: 8
integers3 now contains: 1 2 3 4 5 6 7 1000
```

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Functions outputVector and inputVector
 - Function outputVector uses a range-based for statement to obtain the value in each element of the vector for output.
 - ◆ You could use a counter-controlled loop, but the range-based for is recommended.

```
91 // output vector contents
92 void outputVector(const std::vector<int>& items) {
93     for (const int& item : items) {
94         std::cout << item << ' ';
95     }
96
97     std::cout << '\n';
98 }
```

Objects-Natural Case Study: C++ Standard Library Class Template vector (Cont.)

- Functions outputVector and inputVector
 - Similarly, function inputVector uses a range-based for statement with an int& range variable that can be used to store an input value in the corresponding vector element.

```
100 // input vector contents
101 void inputVector(std::vector<int>& items) {
102     for (int& item : items) {
103         std::cin >> item;
104     }
105 }
```

Objects-Natural Case Study: C++ Standard Library Class Template `vector`.

- Straight-Line Code

- As you worked through this chapter's examples, you saw lots of for loops.
- You may have noticed in our Objects-Natural sections that much of the code that creates and uses objects is [straight-line sequential code](#) with few control statements.
- Working with objects often “flattens” code into lots of sequential function calls.